

# 一致性哈希

分布式缓存动态扩容与高可用系统

从"取模灾难"到"平滑扩容"的阶梯式综合实战

适用对象

大二上学期计算机专业学生

教师：戴波

电子科技大学

团队规模

1人 → 3人

哈希环 · 虚拟节点 · 顺时针查找 · 故障漂移 · 客户端缓存 · 分布式扩缩容

# 真实业务背景

应用场景：某大型互联网公司运营着一个分布式缓存集群（如 Redis Cluster / Memcached），初始含 10 个缓存节点，存储 1 亿条键值对（键为字符串，值为缓存数据）。系统需支持动态扩容、故障容错、低延迟查询和负载均衡。

## 核心挑战

- 动态扩容：新增节点时数据迁移量接近理论下限  $1/N$ （例如在 10 节点以上集群中迁移率  $\leq 10\%$ ），避免大规模迁移导致服务不可用
- 故障容错：节点下线后数据自动分配给其他节点，查询不失效
- 低延迟：单次查找节点跳转次数  $\leq 2$ ，避免跨节点多次网络通信
- 负载均衡：各节点存储键值对数量差异  $\leq 5\%$

## 核心矛盾

### 传统取模灾难：

$\text{hash}(\text{key}) \% N$  在节点数变化时，几乎所有数据 ( $\approx 100\%$ ) 都需要迁移，导致服务不可用。

### 一致性哈希解法：

通过环形空间 + 虚拟节点，节点变化时仅迁移相邻节点数据，迁移率  $\approx 1/N$ 。

# 六阶段阶梯路线图

| 阶段  | 子项目名称          | 对应核心能力           | 难度 | 人数  | 开发周期    |
|-----|----------------|------------------|----|-----|---------|
| 1   | 传统哈希取模与扩容灾难    | 取模映射、迁移率计算       | 2  | 1 人 | 3—4 天   |
| 2   | 一致性哈希基础（无虚拟节点） | 哈希环、顺时针查找        | 4  | 1 人 | 5—7 天   |
| 3   | 虚拟节点与负载均衡      | 虚拟节点、均衡性优化       | 6  | 1 人 | 1—1.5 周 |
| 4   | 故障容错与自动漂移      | 节点下线、数据漂移        | 8  | 2 人 | 1.5—2 周 |
| 5   | 客户端元数据缓存与跳转优化  | 本地缓存、跳转 $\leq 2$ | 10 | 3 人 | 2 周     |
| 6 ★ | 分布式集群与动态扩缩容    | RPC 通信、并发扩容、一致性  | 20 | 3 人 | 3—4 周   |

## 背景

平台初期只有 3 个缓存节点，使用最简单的  $\text{hash}(\text{key}) \% 3$  进行数据分片。  
运营需模拟新增节点时数据迁移量，直观感受"灾难性"迁移。

## 要求

1. 实现简单缓存客户端，支持  $\text{put}(\text{key}, \text{value})$  和  $\text{get}(\text{key})$ ，底层使用  $\text{hash}(\text{key}) \% N$  选择节点（模拟多节点，无需真实网络）
2. 生成 10 万条随机键，初始 3 个节点，统计各节点数据量
3. 增加 1 个节点（变为 4 个），重新计算所有键归属，统计迁移率
4. 对比不同初始节点数和新增数（如  $5 \rightarrow 6$ ,  $10 \rightarrow 11$ ）的迁移率，绘制曲线

## 验收标准

1. 迁移率  $\approx 100\%$ （几乎所有键都需迁移），提交数据证明
2. 提供"节点数变化 vs 迁移率"曲线图
3. 实验报告分析：为何取模扩容会导致服务不可用
4. 代码零第三方库，命令行运行

### 基础水平

编程语言基础、哈希函数（取模、字符串哈希）、基本统计

人数 / 开发时间

1 人 / 3—4 天

## 背景

团队引入一致性哈希解决迁移问题。先实现最简版——仅有物理节点，不引入虚拟节点，观察其负载均衡效果。

## 要求

1. 实现一致性哈希核心数据结构：哈希环（用有序列表或 SortedMap 存储节点哈希值  $\rightarrow$  节点 ID）
2. 实现节点加入、节点移除、数据归属查找（顺时针找到第一个节点）
3. 使用 10 个物理节点，存储 10 万条键，统计每个节点的数据量
4. 增加 1 个节点，统计迁移率（应远小于取模方案）；对比不同节点数量下的负载均衡程度

## 验收标准

1. 新增节点时迁移率  $< 20\%$ （远低于取模的 100%），提交数据
2. 10 个物理节点时，负载不均衡明显（最大/最小负载比  $> 3$ ），需报告此问题
3. 查询正确性验证：随机抽取 1000 个键，归属节点与暴力枚举一致
4. 代码模块化：环管理、节点管理、数据映射分离

### 基础水平

阶段 1 + 有序数据结构（TreeMap / 排序列表 + 二分查找）、哈希函数

人数 / 开发时间

1 人 / 5—7 天

## 背景

阶段 2 发现物理节点少时负载倾斜严重。引入虚拟节点（每个物理节点对应多个虚拟节点）解决均衡问题，需达到差异  $\leq 5\%$  的目标。

## 要求

1. 扩展一致性哈希，为每个物理节点配置  $V$  个虚拟节点（如  $V=100, 200, 500$ ）
2. 虚拟节点命名规则：物理节点ID + "#" + 序号，对其哈希得到环上位置
3. 存储数据时，先找到虚拟节点，再映射到物理节点
4. 测试不同  $V$  值（10, 50, 100, 200）下 10 个物理节点存储 10 万条键的负载均衡。验证新增节点迁移率接近理论下限  $1/N$ （在  $N=10$  时  $\leq 10\%$ ）

## 验收标准

1. 当  $V=200$  时，各物理节点数据量差异  $\leq 5\%$ （提交统计表）
2. 新增节点迁移率  $\leq 10\%$ （在虚拟节点数  $V \geq 100$ 、物理节点数  $N \geq 10$  时验证）
3. 提供"虚拟节点数 vs 负载均衡度"曲线图，分析  $V$  的合理取值。
4. 删除节点测试：节点下线后数据漂移，负载均衡仍保持；文档说明虚拟节点对存储开销的影响

### 基础水平

阶段 2 + 统计知识（均值、标准差）、性能测试方法

人数 / 开发时间

1 人 / 1—1.5 周

## 背景

生产环境中节点故障不可避免。要求节点下线后，其负责的数据自动"漂移"到下一个健康节点，查询不失效，且无人工干预。

## 要求

1. 实现节点故障检测与移除：模拟节点宕机（手动标记下线），从环中移除该物理节点及其所有虚拟节点
2. 实现数据自动漂移：原本属于该节点的键，重新归属到顺时针下一个健康节点
3. 验证"查询不失效"：故障前能查到的键，故障后也能查到（虽然存储位置变了）
4. 实现节点恢复（重新加入）时数据回迁；测试连续多个节点故障（如 2 个、3 个）时系统的可用性

## 验收标准

1. 单节点下线后，迁移率  $\leq 10\%$ （仅该节点数据漂移），提交迁移量报告
2. 随机选取 1000 个键，故障前后都能正确查询到值（一致性验证）
3. 连续 3 个节点故障后，系统仍可正常服务（剩余节点负载均衡差异  $\leq 10\%$ ）
4. 提供故障恢复自动化脚本并报告恢复时间；分析故障期间对查询延迟的影响

### 基础水平

阶段 3 + 异常处理、容错设计思想

人数 / 开发时间

2 人 / 1.5—2 周

## 背景

要求单次查找节点跳转次数  $\leq 2$ 。若每次查询都需在客户端计算哈希并可能跨节点，需将环映射表（元数据）缓存到客户端本地，减少网络跳转。

## 要求

1. 设计客户端本地缓存：存储当前完整的虚拟节点→物理节点映射表（V\*N 个条目，约  $200 \times 10 = 2000$ ，几 KB）
2. 客户端首次启动或感知到集群变化（节点增减）时，从配置中心拉取最新映射表
3. 查询时，客户端本地计算键的哈希，直接在映射表中找到目标物理节点，跳转次数 = 1
4. 若本地映射表过期（如节点已下线），客户端需重试并更新缓存，保证最终一致性；模拟元数据同步延迟场景测试正确性

## 验收标准

1. 常规查询时，跳转次数 = 1（客户端直连目标节点），满足  $\leq 2$  要求
2. 节点扩容/缩容后，客户端能在 **10 秒内**感知并更新本地缓存（可配置）
3. 元数据更新期间，查询可能出现短暂错误，但重试后恢复，提交错误率统计
4. 提供客户端缓存更新策略的对比实验（主动拉取 vs 被动推送）；分析缓存映射表的存储开销和更新开销

### 基础水平

阶段 4 + 网络通信模拟（RPC/HTTP）、缓存更新策略、最终一致性理解

人数 / 开发时间

3 人 / 2 周

## 背景

系统演进为真实分布式环境：多个缓存节点（进程）通过 RPC 通信，数据实际存储在各节点内存中。需实现集群管理、并发扩容、元数据同步的一致性协议（如 Gossip 或 Raft 简化版），并支撑大规模数据。

## 要求

1. 用多进程模拟多个缓存节点（ $\geq 5$  个），每个节点独立存储键值对，提供 RPC 接口（可用 Socket 或 gRPC 模拟）
2. 实现配置中心/协调者（可独立进程），维护一致性哈希环元数据，并通知所有客户端
3. 实现动态扩容：新增节点时协调者更新环，触发数据迁移（实际数据从旧节点复制到新节点）
4. 实现并发扩容：扩容过程中不影响正常读写（一致性快照或双写机制）；实现故障检测：节点心跳机制，超时自动剔除并触发数据漂移；压测 100 万条数据

## 验收标准

1. 扩容过程中，读请求成功率  $\geq 99.9\%$ ，写请求最终一致（提交延迟/成功率数据）
2. 数据迁移量接近理论下限  $1/N$ （ $N$  为扩容前节点数）；当  $N \geq 10$  时，迁移率  $\leq 10\%$  可作为达成指标。
3. 节点故障后，服务恢复时间  $\leq 30$  秒（含检测+漂移）
4. 提交架构设计报告（含元数据同步协议、迁移流程、容错机制）；模拟至少 3 种扩容/缩容场景；最终系统在 10 节点、100 万键下，单次查询平均延迟  $< 10\text{ms}$

### 基础水平

阶段 1—5 + 操作系统/计算机网络（进程通信、socket 编程）、分布式共识基础知识（可自学）

人数 / 开发时间

3 人 / 3—4 周

# 成果交付要求

每阶段必须交付以下成果物，确保过程可追溯、结果可复现。

01 每阶段完整源代码

02 自动化测试脚本（生成键+模拟扩容/故障）

03 性能分析报告（迁移率、均衡度、延迟）

04 架构设计文档（阶段 4/5/6 重点）

05 答辩 PPT

# 能力成长地图

六个阶梯，一次完整的工程能力蜕变。



## 核心收获

- 深入理解一致性哈希原理：哈希环 + 虚拟节点如何最小化数据迁移
- 掌握负载均衡量化分析方法：标准差、最大/最小比、迁移率曲线
- 体验故障容错与自动漂移的工程实现，理解最终一致性模型
- 挑战分布式集群管理：RPC 通信、心跳检测、并发扩容与一致性保障

高难度下，触碰大厂分布式存储团队的核心问题——节点变化时的数据一致性、高可用与低延迟。

祝项目顺利